POOJITH DEVAN

poojithdevan@gmail.com | +91 98183 21997 | linkedin.com/in/poojith-devan | github.com/poojithdevan4D

Profile

I work on running LLMs efficiently on limited hardware — quantization, KV-cache compression, and inference optimization. Most of my work is implementing methods from scratch and benchmarking them on a 4 GB laptop GPU (RTX 3050) and old ARM phones (Cortex-A9, Cortex-A53). Currently interning on 1.58-bit ternary inference at OneBit and on Q1.15 fixed-point inference at Trebuchet System.

Education

O.P. Jindal Global University — M.Sc. in AI and Data Science Jun 2025 – Jun 2026

SRM University — M.C.A. in Generative AI (concurrent) Jun 2025 – Jun 2027

SRM University — B.C.A. in Data Science Jun 2023 – Jun 2025

Projects

Enterprise LLM Task Auditor — M.Sc. Capstone 2026

- Built a tool that answers one practical question for a given task: which open-weight model reliably returns correct JSON, and is it cheaper to self-host or call a cloud API? Generic benchmarks (MMLU, HELM) don't answer that, so it builds a task-specific test set instead.
- Scored 8 candidate models (Qwen, Llama, Gemma, Mistral-Nemo, via OpenRouter) on two independent axes — format reliability (valid JSON with all required keys) and content correctness (per-field similarity to a known-correct answer) — combined so a model has to be both well-formed and right.
- Replaced an earlier TF-IDF cosine score after realising it is degenerate when comparing just two strings (it collapses to shared-token overlap, dominated by the identical JSON keys); switched to per-field `difflib`, which actually measures whether the values are correct.
- Added a cost-of-ownership step that finds the break-even daily volume between an amortised local GPU and live cloud pricing, plus a rule that forces a local-only recommendation for regulated data. Shipped as a CLI and a minimal standard-library web UI with a plain HTML report.

Speculative Decoding from Scratch — Qwen2.5-1.5B on 4 GB GPU May 2026

github.com/poojithdevan4D/qwen-speculative-decoding

- Implemented speculative decoding (Leviathan et al., 2023) from scratch, using Qwen2.5-1.5B (4-bit NF4) as the target and Qwen2.5-0.5B (FP16) as the draft, both running together on one 4 GB RTX 3050 laptop GPU.
- Checked correctness by asserting byte-for-byte identical output to plain greedy decoding across K = 1, 2, 4, 6, 8 — which also confirmed the KV-cache cropping (`DynamicCache.crop`) was right.
- Benchmarked it against HuggingFace's `assisted_generation`: my version reached 1.07x speedup vs their 1.16x. I traced the ∼9% gap to CUDA Graphs and operator fusion that eager-mode PyTorch doesn't have.
- Added streaming token output, a K value that adapts to the measured acceptance rate, and rejection sampling for non-greedy generation.

QwenQuant — 4-Way LLM Quantization Benchmark May 2026

github.com/poojithdevan4D/QwenQuant

- Built a benchmark comparing four ways of running Qwen2.5-1.5B (FP16, INT8 Quanto, INT4 NF4 BitsAndBytes, Q4_K_M GGUF) on a 4 GB RTX 3050.
- Q4_K_M with full GPU offload (llama-cpp-python) gave 6.5x more throughput (16.5 to 107.1 tokens/sec) and used 67% less VRAM.
- Caught a measurement bug before publishing: I had compared perplexity using different tokenizers per backend, which made Q4_K_M look 75% worse than it actually was. Re-ran all four with one tokenizer (INT8 +0.25%, NF4 +6.69%, Q4_K_M +2.61% vs FP16).

- Takeaway: PyTorch-native quantization (Quanto, BitsAndBytes) saves memory but loses throughput on consumer Ampere because there are no fused low-bit GEMM kernels; llama.cpp's K-quants get both.

QuantEdge Phase 1 — BlazeFace on ARM Cortex-A9 2025

- Cross-compiled BlazeFace face detection to an old Samsung GT-S7392 (ARM Cortex-A9, 32-bit, no NEON FP16) with TensorFlow Lite; measured a 92 ms / 10.8 FPS FP32 NEON baseline.
- Hand-wrote ARM NEON INT8 SIMD intrinsics for the convolution hot path, cutting latency to 29.9 ms — 2.5x faster than FP32 NEON on the same phone.
- Wrote up a reproducible benchmark and report. A senior ARM engineer referred me after seeing it.

QuantEdge Phase 2 — TinyLlama 1.1B on Exynos 7870 2025 – 2026

- Ran TinyLlama 1.1B (Q4_K_M GGUF) on a Samsung Galaxy On Nxt (Exynos 7870, Cortex-A53) with llama.cpp; worked around a 32-bit Android userspace running on 64-bit ARMv8 hardware.
- Benchmarked inference across thread counts: 1.83 tokens/sec at 4 threads, with memory bandwidth saturating at 8 threads. I couldn't find a prior public benchmark for this exact chip.
- Wrote a short technical report and methodology slides.

TurboQuant KV Cache Experiment — RTX 3050 2026

- Reproduced TurboQuant (ICLR 2026, arXiv 2504.19874) and added my own KV-cache quantization experiment on consumer Ampere.
- Swept precision settings to find the accuracy/compression trade-off: 3-bit KV quantization kept 80% top-5 recall at 5.3x compression vs an FP16 cache.
- Made an HTML study guide and a technical deck linking the TurboQuant results to my QuantEdge ARM work.

Experience

Edge AI Intern, OneBit Apr 2026 – Present

- Co-authored the Cloe v1 technical report on hybrid surgical distillation for 1.58-bit edge quantization: a hybrid ternary/BF16 Qwen 0.8B reached 44% MMLU, beating full-precision 0.5B and 1B baselines. Published on OneBit's research page.
- Working on 1.58-bit ternary LLM quantization on consumer GPUs (RTX 30-series), focused on keeping accuracy under very low bit-widths.
- Testing custom low-bit accumulation strategies for ternary matrix multiplication on consumer hardware.

Research Intern, Trebuchet System Apr 2026 – Present

- Researching Q1.15 fixed-point arithmetic for inference on low-power hardware that has no floating-point unit.
- Building numerical-stability benchmarks for FP32-to-fixed-point conversion on vision models (ResNet-20 variants on CIFAR-10, run on Modal cloud).

Skills

Languages: Python, C, C++, CUDA C, SQL
ML Frameworks: PyTorch, HuggingFace Transformers, TensorFlow, ONNX, Scikit-learn
Quantization: 1.58-bit ternary, Q1.15 fixed-point, INT4 NF4, INT8, Q4_K_M K-quants, KV-cache quantization, GGUF; bitsandbytes, optimum-quanto, llama.cpp, ggml
Inference Optimization: speculative decoding (from-scratch), DynamicCache management, ARM NEON SIMD intrinsics, llama-cpp-python, TensorFlow Lite, ONNX Runtime, cuBLAS, Nsight; familiar with vLLM, TensorRT-LLM, CUDA Graphs
Hardware: NVIDIA RTX 3050 (Ampere), ARM Cortex-A9, ARM Cortex-A53 (Exynos 7870), Raspberry Pi 4, Jetson Nano
Toolchain: Android NDK, GCC ARM cross-compilation, CMake, Git, Modal cloud, Netron